

Cloud Notes API — Guided Project

Objective

Build and deploy a simple cloud-based Notes API using FastAPI, PostgreSQL, JWT Authentication, GitHub, and Render.

By the end of this project, students will learn:

- How APIs work
 - How authentication works
 - How databases connect to cloud apps
 - How to deploy APIs online
 - How to use environment variables securely
-

Final Features

The API will support:

- User Registration
 - User Login
 - Create Note
 - View Notes
 - Update Note
 - Delete Note
 - Deploy to Cloud
-

Project Structure

Create this folder structure:

```
cloud_notes/  
|— main.py  
|— database.py  
|— models.py  
|— schemas.py  
|— auth.py  
|— requirements.txt  
|— .env
```

Phase 1: Setup (15 mins)

Step 1: Create Project Folder

```
mkdir cloud_notes  
cd cloud_notes
```

Step 2: Create Virtual Environment

Windows:

```
python -m venv venv  
venv\Scripts\activate
```

Mac/Linux:

```
python3 -m venv venv  
source venv/bin/activate
```

Step 3: Install Dependencies

```
pip install fastapi uvicorn sqlalchemy psycopg2-binary python-jose passlib  
bcrypt python-dotenv
```

Save them:

```
pip freeze > requirements.txt
```

Phase 2: Database Setup (15 mins)

Create file:

database.py

Purpose:

- Connect app to PostgreSQL

- Create database session

Students must create:

- engine
- SessionLocal
- Base
- get_db()

Important concept:

Every API request needs database access.

Phase 3: Models (15 mins)

Create:

models.py

Create two tables:

User Model

Fields:

- id
 - name
 - email
 - password
-

Note Model

Fields:

- id
- title
- content
- owner_id

Relationship:

One user → many notes

Important concept:

Foreign key relationship.

Phase 4: Schemas (15 mins)

Create:

schemas.py

Create:

UserCreate

Input for registration.

Fields:

- name
 - email
 - password
-

UserLogin

Input for login.

Fields:

- email
 - password
-

NoteCreate

Fields:

- title
 - content
-

NoteResponse

Output for notes.

Important concept:

Pydantic validates data.

Phase 5: Authentication (20 mins)

Create:

auth.py

Build:

Password Hashing

Functions:

- hash_password()
 - verify_password()
-

JWT Token

Functions:

- create_access_token()
-

Important concept:

Never store plain passwords.

JWT allows secure access.

Phase 6: Main API (30 mins)

Create:

main.py

Import everything.

Create app:

```
app = FastAPI()
```

Create tables:

```
Base.metadata.create_all(bind=engine)
```

Build Endpoints

1. Register

Route:

```
POST /register
```

Flow:

- receive data
- hash password
- save user

Test in Swagger.

2. Login

Route:

```
POST /login
```

Flow:

- verify email
- verify password
- generate token

Return:

```
{  
  "access_token": "..."  
}
```

3. Create Note

Route:

POST /notes

Protected route.

Flow:

- verify JWT
 - create note
 - attach owner_id
-

4. Get Notes

Route:

GET /notes

Return current user's notes.

5. Update Note

Route:

PUT /notes/{id}

Only note owner can update.

6. Delete Note

Route:

DELETE /notes/{id}

Only note owner can delete.

Phase 7: Run Locally (10 mins)

Run:

```
uvicorn main:app --reload
```

Open:

```
http://127.0.0.1:8000/docs
```

Use Swagger UI.

Test:

1. Register
2. Login
3. Copy token
4. Authorize
5. Create notes

Phase 8: Push to GitHub (10 mins)

Initialize git:

```
git init
git add .
git commit -m "Cloud Notes API"
```

Create repository on GitHub.

Push:

```
git remote add origin YOUR_REPO_URL
git push -u origin main
```

Phase 9: Cloud Database (10 mins)

Go to:

Neon PostgreSQL

Create:

- Project
- Database

Copy:

DATABASE_URL

Put into:

.env

Example:

```
DATABASE_URL=your_database_url  
SECRET_KEY=your_secret_key
```

Important:

Never hardcode secrets.

Phase 10: Deploy on Render (20 mins)

Go to Render.

Steps:

1. New Web Service
2. Connect GitHub repo
3. Select repository
4. Add environment variables
5. Deploy

Start command:

```
uvicorn main:app --host 0.0.0.0 --port 10000
```

Final Testing

After deployment:

Open:

```
https://your-app-url.onrender.com/docs
```

Test:

- Register
- Login
- Create Notes
- View Notes
- Update Notes
- Delete Notes

Everything should work live.

Submission Requirements

Students must submit:

- GitHub Repository Link
 - Live Render Link
 - Screenshots of Swagger API
 - Screenshot of Neon Database tables
-

Bonus Tasks

If finished early:

1. Add created_at field
 2. Add note search
 3. Add note categories
 4. Add pagination
 5. Add public/private notes
-

Learning Outcomes

After this project students will understand:

- FastAPI basics
- REST APIs
- SQLAlchemy ORM
- PostgreSQL cloud databases
- JWT authentication
- Environment variables
- Cloud deployment
- GitHub workflow

This is the foundation of modern backend cloud development.